

Cấu trúc dữ liệu và giải thuật - IT003.P21.CTTN

Assignment 4.02

Lê Hữu Hoàng - 24520539 - ATTN2024

Cho một cây nhị phân, giả sử nếu đặt 01 **camera** vào một node thì sẽ quan sát được bản thân nó và node cha cùng 2 node con (nếu có - tức mỗi camera có thể nhìn thấy từ 1 đến 4 node). Viết thuật toán và hàm xác định **số camera tối thiểu** cần dùng để có thể thấy toàn bộ các node trong cây.

Ý tưởng: sử dụng duyệt cây theo LRN và Tham lam theo đệ quy

Trước tiên, ta phân loại các kiểu đánh dấu:

- 0: node chưa được quan sát và cần được quan sát bởi node cha
- 1: node đã được camera quan sát, nhưng không phải chính nó
- 2: node có 1 camera tại chính nó

```

1  #include <iostream>
2  using namespace std;
3
4  int count=0;
5  struct treenode {
6      int val;
7      treenode* left;
8      treenode* right;
9      treenode(int x): val(x), left(NULL), right(NULL) {}
10 };
11
12 int dfs(treenode* node, int& Count) {
13     if (!node) return 2; // Nếu node rỗng, coi như đã bị quan sát
14
15     int left = dfs(node->left, count);
16     int right = dfs(node->right, count);
17
18     // Nếu cả hai node con đều không có camera và không bị quan sát, cần ph
19     if (left == 0 || right == 0) {
20         count++;
21         return 1; // Đặt camera tại node hiện tại
22     }
23
24     // Nếu một trong hai node con có camera, node này sẽ bị quan sát
25     if (left == 1 || right == 1)
26         return 2; // Node này bị quan sát
27
28
29     // Nếu cả hai node con đều bị quan sát, node này không cần camera
30     return 0; // Node này không bị che khuất và không cần camera
31 }
32
33 int main() {
34
35     treenode* root=new treenode(8);
36     root->left=new treenode(2);
37     root->left->left= new treenode(1);
38     root->left->right= new treenode(5);
39     root->left->right->left=new treenode(3);
40     root->right=new treenode(10);
41     root->right->left=new treenode(9);
42     root->right->right=new treenode(12);
43     root->right->right->right=new treenode(14);
44
45     // Nếu root không bị quan sát, cần phải đặt camera ở root
46     if (dfs(root, count) == 0)
47         count++;
48     cout << count;
49
50     return 0;
51 }
52

```

```
int dfs(treenode* node, int& Count) {
    if (!node) return 2; // Nếu node rỗng, coi như đã bị quan sát

    int left = dfs(node->left, count);
    int right = dfs(node->right, count);

    // Nếu cả hai node con đều không có camera và không bị quan sát, cần phải đặt camera tại node này
    if (left == 0 || right == 0) {
        count++;
        return 1; // Đặt camera tại node hiện tại
    }

    // Nếu một trong hai node con có camera, node này sẽ bị quan sát
    if (left == 1 || right == 1)
        return 2; // Node này bị quan sát

    // Nếu cả hai node con đều bị quan sát, node này không cần camera
    return 0; // Node này không bị che khuất và không cần camera
}
```